Neighbor Detection in OLSRv1 Is Governed by Two Different Timers: Timestamp Evidence from ns-3 and a Decoupled Hold-Time Configuration

[Authors and affiliations to be completed]

Draft manuscript, 19 October 2026

Abstract

In OLSRv1 the validity time advertised in a HELLO message is tied to the HELLO interval (three times the interval in the RFC 3626 recommendation and in the ns-3 model). Using timestamp-level traces in ns-3.49 we show that this single coupling hides two different timers. A departed neighbour is purged exactly when the validity time of the last HELLO received from it expires (observed minus predicted purge time: 0.03 s on average and at most 0.05 s, the polling resolution; 1,440 observations), so departure detection takes about hold time minus half the HELLO interval. Discovery of a returning neighbour takes about 1.5 HELLO intervals and does not depend on the hold time. Accelerating HELLO transmission after a topology event therefore cannot shorten the detection of a departure, which is consistent with a frozen 600-run confirmatory study in which a gap-triggered adaptive HELLO controller did not improve on stock OLSR. Decoupling the two parameters (HELLO every 3 s, hold 6 s) was evaluated in a pre-specified 900-run study (6 timer settings, 5 scenarios, 3 network sizes, 10 independent seeds). Relative to stock OLSR (HELLO 2 s, hold 6 s) it reduced OLSR control bytes by about 10% under churn and node outage and by 7% in stable networks, with delivery ratio equal or higher (churn: +2.7 percentage points, 95% CI +0.3 to +5.9, exploratory). The setting should not be used on fading channels, where it lowered delivery ratio by 8 to 9 points. We release the code, the one-attribute ns-3 patch, the protocol and all raw results.

Keywords: OLSR, HELLO interval, neighbour hold time, MANET, ns-3, neighbour discovery.

1 Introduction

Proactive MANET routing protocols such as OLSRv1 detect link changes through periodic HELLO messages. Each HELLO carries a validity time (VTime) that tells receivers how long to keep the sender as a neighbour. RFC 3626 recommends a neighbour hold time of three HELLO refresh intervals, and the ns-3 OLSR model derives the advertised validity time from the HelloInterval attribute in the same way. One parameter therefore controls both how many control bytes a node sends and how long stale neighbour state survives.

A natural idea is to adapt the HELLO rate: send faster when the neighbourhood seems to be changing and slower when it is stable. We tested this idea first, with a controller that reacts to a gap in expected HELLO arrivals, in a frozen 600-run confirmatory experiment. It did not improve delivery ratio over stock OLSR. This paper explains why, and tests the alternative the explanation suggests.

Contributions:

1. A two-timer model, validated with timestamps, in which departure detection is governed by the hold time and arrival detection by the HELLO interval.

2. An explanation of why sender-side HELLO acceleration cannot speed up departure detection, together with the earlier negative confirmatory result.
3. A pre-specified evaluation (900 runs) of decoupled settings, with the conditions under which they help (stable, churn, outage) and fail (fading).
4. Released artefacts: simulation code, a one-attribute ns-3 patch, the protocol written before the runs, and all raw results including null and negative ones.

2 Background and related work

In OLSRv1 (RFC 3626) a node learns symmetric neighbours from HELLO messages. A link tuple expires when the validity time carried by the last HELLO from that neighbour runs out. The validity time is a field of the message, so the receiver follows what the sender advertises. The recommended value is three times the HELLO refresh interval, but nothing in the message format ties the two together. The later neighbourhood discovery protocol NHDP (RFC 6130) treats the HELLO interval and the hold times as separate parameters. In the ns-3.49 OLSR model the advertised neighbour validity is three times the current HelloInterval attribute, and HELLO emission is jittered.

Adaptive HELLO and neighbour-discovery schemes for ad hoc routing have been proposed in the literature. [The related-work survey and citations are to be completed by the authors after a literature search; no references are asserted in this draft.] Our contribution is not HELLO tuning itself, but the controlled separation of departure and arrival detection with timestamp-level evidence, and the measured trade-off of decoupling the two timers in a real protocol implementation.

3 A two-timer model

Let h be the HELLO interval, V the advertised validity time (hold time), and $k = V / h$ the hold factor (stock OLSR: $k = 3$).

Departure. A neighbour leaves at time t_{dep}. The last HELLO a receiver heard from it was sent at $t_{last} = t_{dep} - a$, where the age a is between 0 and about h, with mean about $h/2$ when the departure time is not aligned with the HELLO timer. The receiver purges the neighbour at $t_{last} + V$, regardless of anything the departed node does afterwards. The detection delay is therefore $V - a$, with mean about $V - h/2$.

Arrival. A node returns at a random time. Its first HELLO after the return arrives after about $h/2$ on average. The receiver lists it as symmetric only after a further HELLO from that node that includes the receiver in its heard list, about one more interval. Mean arrival delay is therefore about 1.5 h, independent of V.

Consequences. (i) Once a neighbour has left, no change in its sending rate can move the purge time. Sender-side acceleration can only shorten the age a of the last HELLO before the event. (ii) With the stock coupling $V = 3h$, lowering h to find arrivals faster also lowers V, and raising h to save bytes also raises V. (iii) A hold factor below about 3 shortens departure detection but makes false purges more likely, because fewer consecutive HELLO losses are needed to expire a live neighbour.

4 Method

4.1 Simulation setup

All experiments use ns-3.49 (OLSRv1 model, IEEE 802.11b at 1 Mbit/s, ad hoc mode) on grid topologies with 35 m spacing and a 65 m maximum range. Path loss is log-distance (exponent 3.0, or 3.5 with Nakagami fading in the harsh scenarios, reference loss 40.05 dB). The OLSR TC interval is 5 s. Application traffic is up to five UDP flows (256-byte payload every 0.25 s) between fixed node pairs.

Measurements cover 25 s to 90 s; topology changes occur at 45 s and are reverted at 70 s. Every run uses a separate process, fixed random-stream assignment on the Wi-Fi devices, channel and OLSR, and identical traffic and topology schedule across timer settings.

4.2 Timer settings

Six static settings, written H(hello)/hold: stock default H2/6 s, H3/6 s, H3/7.5 s, H3/9 s, H5/12.5 s and stock slow H5/15 s. The three settings with a hold factor other than 3 use a one-attribute patch (NeighbHoldFactor) that makes the advertised validity time a configurable multiple of the HELLO interval. With the default factor of 3 the model behaves as stock ns-3.

4.3 Experiment A: timestamp trace (mechanism)

A 5 by 5 grid runs OLSR control traffic only, with no data traffic and no fading. The central node (eight in-range neighbours) leaves the network at a time drawn between 40 s and 70 s and returns 30 to 60 s later; times are generated from low-discrepancy sequences of the seed and are identical across settings, so they are not aligned with the HELLO timer grid. Each neighbour logs the last HELLO received before the departure and its advertised validity time, polls its symmetric neighbour set every 50 ms, and records the purge time, the first HELLO after the return, and the time the returning node is listed again. 6 settings by 30 seeds (4001 to 4030) gave 1,440 observer records. The unit of analysis for means and confidence intervals is the seed (the eight observers of one run share the same leaver and timing).

4.4 Experiment B: system-level study

Pre-specified in a protocol written before any run: 6 settings by 5 scenarios (stable, fading, churn, outage, stress) by 3 sizes (25, 50, 100 nodes) by 10 new seeds (2003, 2011, 2017, 2027, 2029, 2039, 2053, 2063, 2069, 2081) = 900 runs. Churn moves every seventh node by (70 m, 25 m); outage moves every thirteenth node 2,000 m away (node 0, a flow source, is isolated for every setting); stress combines both with fading. Outcomes: packet delivery ratio (PDR, attempt-based), OLSR control bytes per node per minute, and an evaluator-only count of stale and missing symmetric-neighbour entries based on node positions. Comparisons are paired against stock default by seed and size, with a bootstrap over seeds for 95% intervals.

4.5 Experiment C: earlier adaptive controller (frozen confirmatory study)

Four policies, 5 scenarios, 3 sizes, 10 seeds (600 runs): stock default (HELLO 2 s), stock slow (5 s), a pilot that reacts to neighbour-set changes, and a gap-hysteresis controller that switches from 5 s to 2 s HELLO for 12 s (cooldown 15 s) when a neighbour's HELLO gap exceeds two expected periods plus 0.35 s. The protocol was frozen before the run and not tuned afterwards.

4.6 Reporting rules

All runs of every experiment are reported, including negative and null results. Parameters were not changed after seeing results within an experiment; a change started a new round with new seeds. About 50 paired comparisons were made in Experiment B, so intervals that exclude zero are treated as exploratory. An early version of the trace used a fixed departure time aligned with the HELLO timer grid and gave arrival delays that were nearly constant across seeds; this phase locking was removed by randomising event times, and only the randomised trace is reported.

5 Results

5.1 Experiment A: the purge time is fixed by the last HELLO

Across 1,440 observations the observed purge time equals the predicted expiry (last HELLO reception time plus the advertised validity time) with a mean difference of 0.025 to 0.031 s per setting and a maximum of 0.050 s, equal to the polling resolution (Figure 1a). The advertised validity was exactly 6, 7.5, 9, 12.5 and 15 s for the corresponding settings. The mean detection delays follow the model (Table 1, Figures 1b and 1c).

Table 1. Mean detection delays over 30 seeds (95% bootstrap CI in brackets) against the two-timer model.

Setting	Hello (s)	Hold (s)	Purge delay (s)	Model: hold − hello/2	Arrival delay (s)	Model: 1.5 × hello
H2/6 (stock)	2	6	4.98 [4.76, 5.20]	5.0	3.07 [2.91, 3.23]	3.0
H3/6	3	6	4.44 [4.09, 4.78]	4.5	4.68 [4.42, 4.94]	4.5
H3/7.5	3	7.5	6.01 [5.66, 6.36]	6.0	4.66 [4.39, 4.92]	4.5
H3/9	3	9	7.42 [7.07, 7.77]	7.5	4.60 [4.36, 4.84]	4.5
H5/12.5	5	12.5	9.88 [9.33, 10.41]	10.0	7.52 [7.12, 7.91]	7.5
H5/15 (stock slow)	5	15	12.38 [11.82, 12.93]	12.5	7.61 [7.19, 8.01]	7.5

A least-squares fit over the 180 seed-level means gives purge delay = 0.06 + 0.996 × hold − 0.526 × hello and arrival delay = 0.16 − 0.013 × hold + 1.519 × hello. The coefficients are close to the model values (1, −0.5) and (0, 1.5). At equal hold time (6 s), the 3 s HELLO setting purges a departed neighbour 0.55 s earlier than the 2 s setting (paired difference, CI −0.89 to −0.20) but takes 1.62 s longer to list a returning neighbour (CI 1.33 to 1.90).

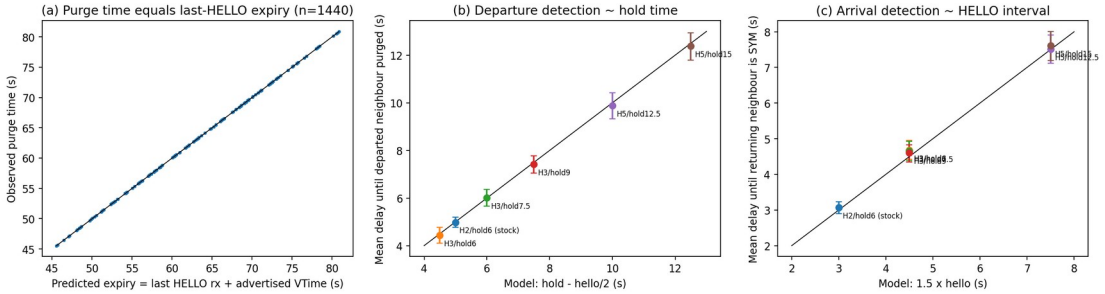


Figure 1. *(a) Observed against predicted purge time. (b) Purge delay against hold − hello/2. (c) Arrival delay against 1.5 × hello. Error bars are 95% bootstrap intervals over seeds.*

5.2 Experiment C: why sender-side adaptation did not help

In the 600-run confirmatory study the gap-hysteresis controller did not improve on stock OLSR (Table 2). Under churn its PDR (76.5%) was close to the stock slow setting (75.6%), which it resembles whenever it is not in fast mode, and 10 points below stock default (86.5%). The controller switched to fast mode

18,172 times in total, but the model above explains the outcome: after a neighbour has left, receivers purge it by the validity time of its last HELLO, which was advertised at the slow setting.

Table 2. Experiment C, mean over all scenarios, sizes and seeds.

Policy	PDR (%)	Control bytes per node per min	PDR under churn (%)
Stock default (H2)	66.5	9,948	86.5
Stock slow (H5)	63.0	8,756	75.6
Pilot	63.1	9,440	77.6
Gap hysteresis	61.7	8,894	76.5

5.3 Experiment B: decoupling hold time from HELLO rate

All 900 runs were valid. The time that stale symmetric entries survive follows the hold time, not the HELLO interval: under churn the mean stale entry-seconds were 672 (hold 6 s, H2), 632 (hold 6 s, H3), 892 (7.5 s), 1,094 (9 s), 1,213 (12.5 s) and 1,578 (15 s); the same ordering appears under outage and stress.

Table 3 compares H3/6 s (same hold as stock default, one third fewer HELLO messages) with stock default. Stock default PDR (pooled) was 98.3% (stable), 34.2% (fading), 86.0% (churn), 86.7% (outage) and 23.8% (stress).

Table 3. H3/6 s minus stock default (H2/6 s), paired by seed and size; 95% CI over seeds in brackets.

Scenario	PDR difference (pp)	Control bytes difference (%)
Stable	+0.9 [−1.1, +3.1]	−7.4
Churn	+2.7 [+0.4, +5.9]	−10.5
Outage	+1.0 [−0.4, +2.6]	−10.0
Fading	−8.6 [−11.0, −6.4]	−33.8
Stress	−7.9 [−9.6, −6.3]	−39.7

In stable, churn and outage the decoupled setting saves 7 to 10% of control bytes without a measurable loss of delivery ratio. Under churn the gain in PDR depends on network size: −1.4 pp at 25 nodes (higher in 2 of 10 seeds), +4.4 pp at 50 nodes (8 of 10) and +5.1 pp at 100 nodes (9 of 10); the byte saving falls from 17.8% at 25 nodes to 4.7% at 100 nodes. Under outage the PDR differences are −0.1, +1.6 and +1.5 pp at 25, 50 and 100 nodes. On fading channels (fading, stress) the same setting lowers PDR by 8 to 9 points. The large byte reductions there (34 to 40%) accompany lost links, not efficiency: fewer neighbours are listed and a hold factor of 2 expires a live neighbour after a single lost HELLO.

Slower stock settings save more bytes than H3/6 s in stable networks (H5/15 s: −17%), but under churn they lose 11.4 pp of PDR (CI −13.2 to −8.8), so they lie on a worse trade-off than the decoupled setting in that regime. Intermediate decoupled settings (H3/7.5 s, H3/9 s, H5/12.5 s) are reported in Appendix A.

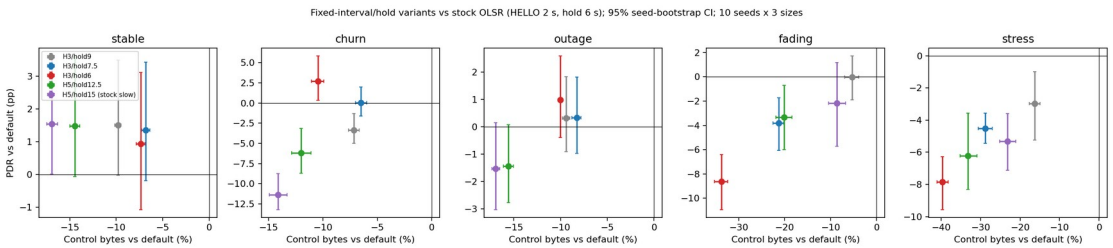


Figure 2. *PDR difference against control-byte difference relative to stock default for each setting and scenario, with 95% bootstrap intervals.*

6 Discussion

The two events a HELLO mechanism must detect, departure and arrival, are served by different timers. Departure is bounded by the hold time carried in the last HELLO, so it is a receiver-side expiry that the departing node cannot influence once it has gone. Arrival is bounded by how often a new node announces itself. With the stock coupling a designer has one dial for both. In the churn and outage regimes tested here, a setting with the stock hold time and a lower HELLO rate keeps departure detection as fast as stock (0.55 s faster) at the price of slower arrival discovery (1.6 s slower) and fewer bytes.

This changes how HELLO adaptation should be designed. A sender cannot speed up departure detection of itself; useful adaptation must act on the receiver's expiry, for example by advertising a validity time that reflects the link quality the receiver sees, or by using link-layer failure feedback. The decoupled setting evaluated here is a static first step. Its failure on fading channels shows the limit: a hold factor near 2 needs a reliable channel, and it would have to be raised when losses are high.

7 Limitations and threats to validity

- Simulation only; one grid family, 802.11b at 1 Mbit/s, five flows, and a single leaver in the trace experiment. Results may differ for other topologies, mobility models and radios.
- Control bytes are OLSR packet bytes, not airtime or energy. Byte savings shrink with network size (17.8% to 4.7% under churn).
- About 50 paired comparisons were made in Experiment B; positive PDR differences are exploratory and several intervals include zero.
- The decoupled settings need a one-attribute patch to the stock model. Interoperability with unmodified receivers was not evaluated; receivers that honour the validity-time field should follow the advertised value, but this was not tested.
- Experiment C tested one adaptive controller design. The result does not rule out every adaptive scheme, in particular receiver-side adaptation.
- Node 0 is isolated for every setting in the outage and stress scenarios, which caps PDR but does not bias the paired comparison.
- Staleness and missing-entry counts use evaluator-only ground truth from node positions, and the missing-entry count is not meaningful under fading.
- Related-work coverage in this draft is incomplete (see Section 2).

8 Conclusion

Timestamp traces show that, in OLSRv1, departure detection is set by the hold time advertised in the last HELLO (delay about hold − hello/2), while arrival detection is set by the HELLO interval (delay about 1.5 × hello). This explains why a sender-side adaptive HELLO controller did not help in a frozen confirmatory study, and it identifies a design space the stock 3× coupling hides. A static decoupled setting

(HELLO 3 s, hold 6 s) reduced control bytes by 7 to 10% against stock OLSR in stable, churn and outage conditions with equal or higher delivery ratio, and degraded on fading channels. Receiver-side adaptation of the hold time is the natural next step.

Reproducibility and data availability

The following files are provided: son_study2.cc (system-level study), son_trace.cc (trace experiment), olsr_patch_NeighbHoldFactor.txt (patch), run_all.sh and run_trace.sh (drivers), SON_V2_Protocol.txt (protocol written before the runs), son_ns3_v2_results.csv (900 runs), son_trace_results_1.csv (1,440 observer records) and the confirmatory data of Experiment C (son_ns3_confirmatory.csv). Each run is a separate ns-3.49 process with seed and run number fixed. The ns-3 commit and build profile (optimized, MSYS2 UCRT64 on Windows) should be added to the final version.

Appendix A: paired results against stock default (all settings, all scenarios)

PDR difference in percentage points and control-byte difference in percent, pooled over sizes; 95% bootstrap CI over seeds for PDR.

Scenario	Setting	PDR diff (pp) [95% CI]	Bytes diff (%)
Stable	H3/9	+1.50 [−0.03, +3.49]	−9.8
Stable	H3/7.5	+1.35 [−0.20, +3.43]	−6.8
Stable	H3/6	+0.93 [−1.08, +3.11]	−7.4
Stable	H5/12.5	+1.48 [−0.06, +3.58]	−14.5
Stable	H5/15	+1.54 [+0.01, +3.62]	−16.9
Churn	H3/9	−3.35 [−4.98, −1.29]	−7.2
Churn	H3/7.5	+0.04 [−1.62, +1.98]	−6.5
Churn	H3/6	+2.71 [+0.35, +5.86]	−10.5
Churn	H5/12.5	−6.16 [−8.69, −3.15]	−12.0
Churn	H5/15	−11.35 [−13.22, −8.75]	−14.1
Outage	H3/9	+0.32 [−0.91, +1.82]	−9.4
Outage	H3/7.5	+0.33 [−0.97, +1.82]	−8.3
Outage	H3/6	+0.98 [−0.40, +2.59]	−10.0
Outage	H5/12.5	−1.44 [−2.78, +0.07]	−15.6
Outage	H5/15	−1.54 [−3.04, +0.15]	−16.9
Fading	H3/9	−0.01 [−1.90, +1.73]	−5.4
Fading	H3/7.5	−3.82 [−6.04, −1.71]	−21.3
Fading	H3/6	−8.62 [−10.95, −6.39]	−33.8
Fading	H5/12.5	−3.33 [−6.00, −0.72]	−20.2
Fading	H5/15	−2.18 [−5.71, +1.19]	−8.6
Stress	H3/9	−2.97 [−5.25, −0.99]	−16.4
Stress	H3/7.5	−4.51 [−5.45, −3.57]	−28.8
Stress	H3/6	−7.86 [−9.58, −6.28]	−39.7
Stress	H5/12.5	−6.22 [−8.32, −3.58]	−33.2
Stress	H5/15	−5.32 [−7.13, −3.58]	−23.2